

ChronoLog

A **distributed shared tiered log** with data ordered by **physical time**, capturing **activity data** from the edge to HPC, and serving as **memory substrate** for **AI agents**



AUTHORS

Eenko Gonzalez
GNOSIS RESEARCH CENTER

Kun Feng
GNOSIS RESEARCH CENTER

Inna Brodtkin
UNIVERSITY OF CHICAGO

Kyle Chard
UNIVERSITY OF CHICAGO

Anthony Kougas
GNOSIS RESEARCH CENTER

Xian-He Sun
GNOSIS RESEARCH CENTER

01 Abstract

Modern systems generate data at unprecedented rates exceeding TB/s: sensors, scientific instruments, IoT devices, and human activity across web, mobile, and edge.

Beyond conventional storage, an increasingly prevalent need is the capture of *activity data*: events as they occur, rather than static state. Domains from scientific computing to AI depend on high-performance logging that existing solutions cannot deliver at scale.

ChronoLog is a scalable distributed log store for this regime, from edge to HPC. It guarantees ordering by physical time, employs multiple storage tiers, supports concurrent access, and enables range queries over partial logs.

02 Key Contributions

Four design principles distinguish ChronoLog from prior log stores, each addressing a specific bottleneck in distributed activity capture at scale.

- **Physical time** — orders events by wall clock, no synchronizations
- **Lock-free append** — avoids slow tail finding caused by protecting locks
- **Auto-tiered** — stores data in best tiers in memory and storage hierarchy
- **Range queries** — retrieves using time range, perfect for time series data

03 Data Model

ChronoLog organizes activity data in a three-level hierarchy:

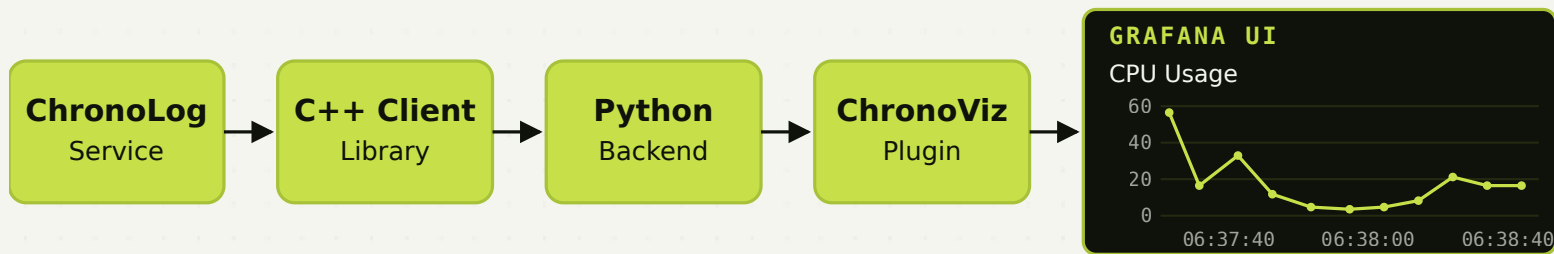
- **Chronicles** partition the namespace, like directories
- **Stories** carry time-ordered streams of data, like files
- **Events** hold individual data records, like lines



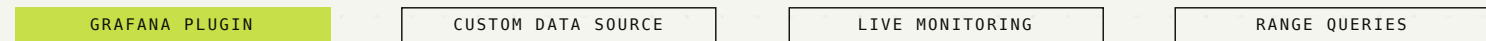
06 Use Cases

CASE A • OBSERVABILITY — CHRONOVIZ

Live HPC Telemetry through a Grafana plugin



Telemetry (CPU, RAM, network) is logged distributedly into ChronoLog and rendered live in Grafana through a native ChronoViz plugin backed by a FastAPI service. Operators inspect arbitrary time ranges through concurrent range queries served across all tiers — no synchronization stall, no centralized sequencer in the path.



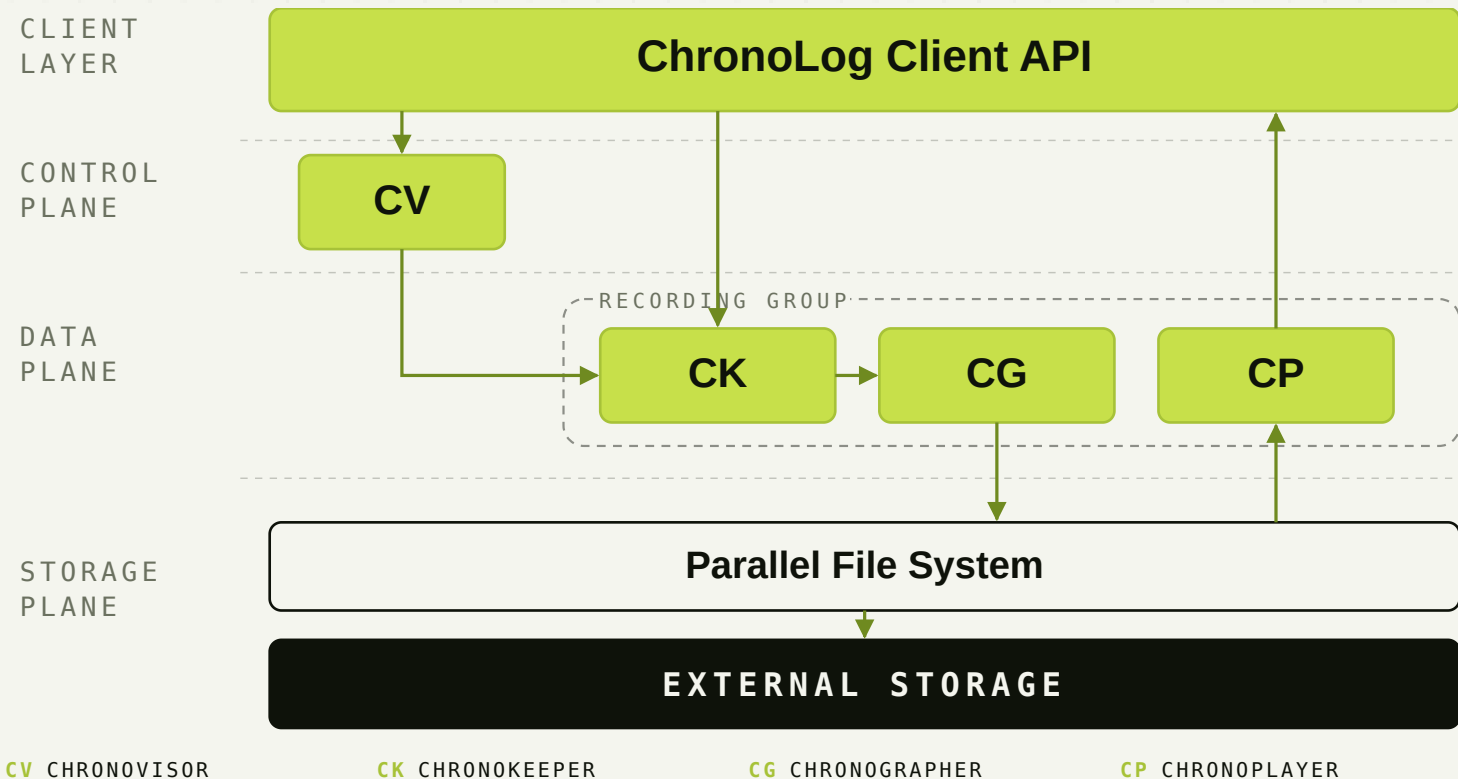
07 On-going and Future Work

ON-GOING

- Multi-channel releases: tarball, docker image, Chameleon image
- Wide Linux distro support: pre-built DEB/RPM packages per release
- Well-maintained documentation: versioned, step-by-step user guide

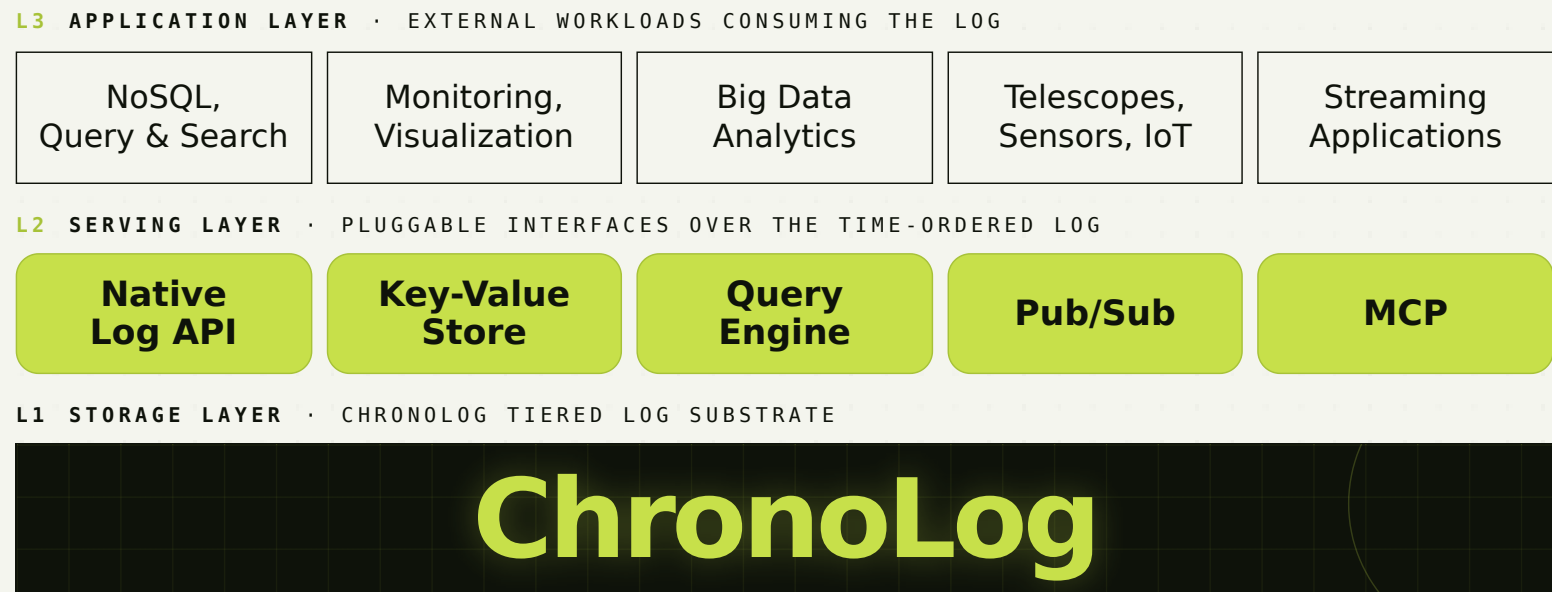
04 Architecture

Four planes separate concerns: clients enter through a single API; ChronoVisor coordinates; recording groups ingest and structure events; data persists to external storage via PFS.



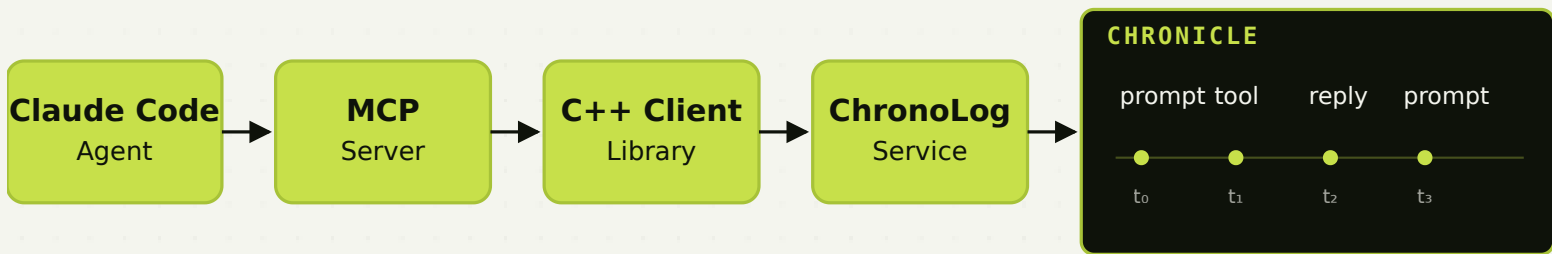
- **ChronoVisor** — Client and component registry, metadata
- **ChronoKeeper** — Hot-tier ingestion via RDMA
- **ChronoGrapher** — Builds stories from chunks, writes to PFS
- **ChronoPlayer** — Replays events in range across all tiers

05 Ecosystem

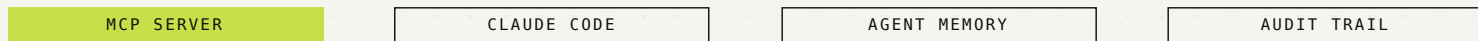


CASE B • AGENT MEMORY — CHRONOLOG MCP

Episodic memory for LLM-based agents via MCP



The ChronoLog MCP server persists and replays LLM agent interactions — public Claude support available today — logging each as a timestamped, ordered event. The shared log becomes a durable substrate for session replay, multi-session context resumption, and audit of long-running design discussions.



REFERENCE

A. Kougas, H. Devarajan, K. Bateman, J. Cernuda, N. Rajesh, X.-H. Sun. “ChronoLog: A Distributed Shared Tiered Log Store with Time-based Data Ordering.” Proceedings of the 36th International Conference on Massive Storage Systems and Technology (MSST 2020).

ACKNOWLEDGMENTS

This work is conducted at the **Gnosis Research Center** in collaboration with the **University of Chicago**, and is supported by the **National Science Foundation** under grant **CSSI-2104013**.

FUNDED BY

